

```
In [2]: # 1(a)
import numpy as np
rng = np.random.default_rng(650)
n = 100
B = rng.standard_normal((n, n))
D = np.diag(np.exp(rng.standard_normal(n)))
Q = B @ B.T + D
b = 10 * rng.standard_normal(n)
```

```
In [3]: # 1(b)
x_star = np.linalg.solve(Q, -b)
gradient_at_x_star = Q @ x_star + b
print("Gradient norm at x_star:", np.linalg.norm(gradient_at_x_star))
```

Gradient norm at x_star: 5.997081613800755e-13

```
In [4]: # helpers
def f(x, Q, b):
    return 0.5 * x @ Q @ x + b @ x

def grad_f(x, Q, b):
    return Q @ x + b
```

```
In [ ]: # 1(c) i. exact line search
def gd_exact(Q, b, x0, num_iters):
    x = np.array(x0, dtype=float, copy=True)

    for _ in range(num_iters):
        gradient = grad_f(x, Q, b)
        if np.all(gradient == 0):
            break
        gradient_squared = gradient @ gradient # numerator
        directional_curvature = gradient @ Q @ gradient # denominator
        step = gradient_squared / directional_curvature
        x = x - step * gradient # descent

    return x
```

```
In [6]: # 1(c) ii. Armijo
def gd_armijo(Q, b, x0, num_iters,
              alpha_bar=1.0, beta=0.5, sigma=1e-4):

    x = np.array(x0, dtype=float, copy=True)

    for _ in range(num_iters):
        gradient = grad_f(x, Q, b)
        if np.all(gradient == 0):
            break
        step = alpha_bar
        current_value = f(x, Q, b)

        while True:
            trial_point = x - step * gradient # descent
            trial_value = f(trial_point, Q, b) # LHS
```

```

        required_value = current_value - sigma * step * (gradient @ grad
        if trial_value <= required_value:
            break
        step *= beta # *= diminishing rate

    x = x - step * gradient # descent

    return x

```

```

In [7]: # 1(c) iii. Diminishing Step Size
def gd_diminishing(Q, b, x0, num_iters, alpha):
    x = np.array(x0, dtype=float, copy=True)

    for r in range(1, num_iters + 1):
        gradient = grad_f(x, Q, b)
        step = alpha / r
        x = x - step * gradient

    return x

```

```

In [ ]: # 1(c) iv. Lipschitz Constant
def gd_constant(Q, b, x0, num_iters):
    x = np.array(x0, dtype=float, copy=True)
    L = np.linalg.eigvalsh(Q).max()
    step = 1.0 / L

    for _ in range(num_iters):
        gradient = grad_f(x, Q, b)
        x = x - step * gradient

    return x

```

```

In [9]: # 1(d) Nesterov's Accelerated Gradient
def nesterov(Q, b, x0, num_iters):
    eigvals = np.linalg.eigvalsh(Q)
    mu = eigvals.min()
    L = eigvals.max()
    kappa = L / mu
    gamma = (np.sqrt(kappa) - 1) / (np.sqrt(kappa) + 1)

    x = np.array(x0, dtype=float, copy=True)
    y = x.copy()

    for _ in range(num_iters):
        y_next = x - grad_f(x, Q, b) / L
        x_next = y_next + gamma * (y_next - y)
        y = y_next
        x = x_next

    return y

```

```

In [10]: # 1(e) Newton's Method
def newton(Q, b, x0, num_iters):
    x = np.array(x0, dtype=float, copy=True)

```

```

for _ in range(num_iters):
    gradient = grad_f(x, Q, b)
    direction = np.linalg.solve(Q, -gradient)
    x = x + direction

return x

```

1(f) Log-relative accuracy after 1000 iterations

Exact line search: -6.507

Armijo: -6.937

Diminishing: -0.037

Constant 1/L: -3.318

Nesterov: -33.374

Newton: -33.071

1(g) Average log-relative accuracy over five realizations

Exact line search: -6.677

Armijo: -7.408

Diminishing: -0.037

Constant 1/L: -3.432

Nesterov: -33.079

Newton: -33.043

1(h) Condition numbers of the five realizations

1: $\kappa = L/\mu = 361.959$

2: $\kappa = L/\mu = 328.380$

3: $\kappa = L/\mu = 384.156$ 4: $\kappa = L/\mu = 387.830$

5: $\kappa = L/\mu = 300.039$

```

In [11]: # 2. Effect of condition number
D = np.diag(np.log(10 + np.exp(rng.standard_normal(n))))

```

log-relative accuracy after 1000 iterations (average over 5 realizations)

Exact line search: -13.620

Armijo: -15.020

Diminishing: -0.072

Constant 1/L: -6.837

Nesterov: -33.523

Newton: -33.430

Condition numbers per trial

1: $\kappa = L/\mu = 164.418$

2: $\kappa = L/\mu = 151.577$

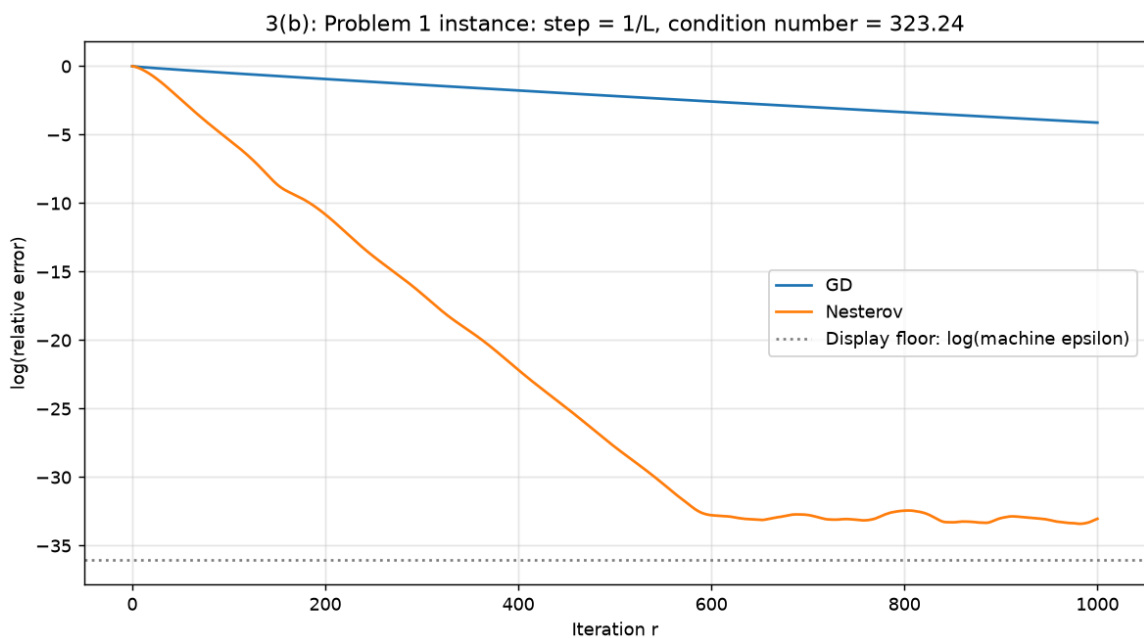
3: $\kappa = L/\mu = 159.288$

4: $\kappa = L/\mu = 160.249$

5: $\kappa = L/\mu = 155.039$

```
In [13]: # 3(a)  
## see 1(a)
```

3(b)



3(c)

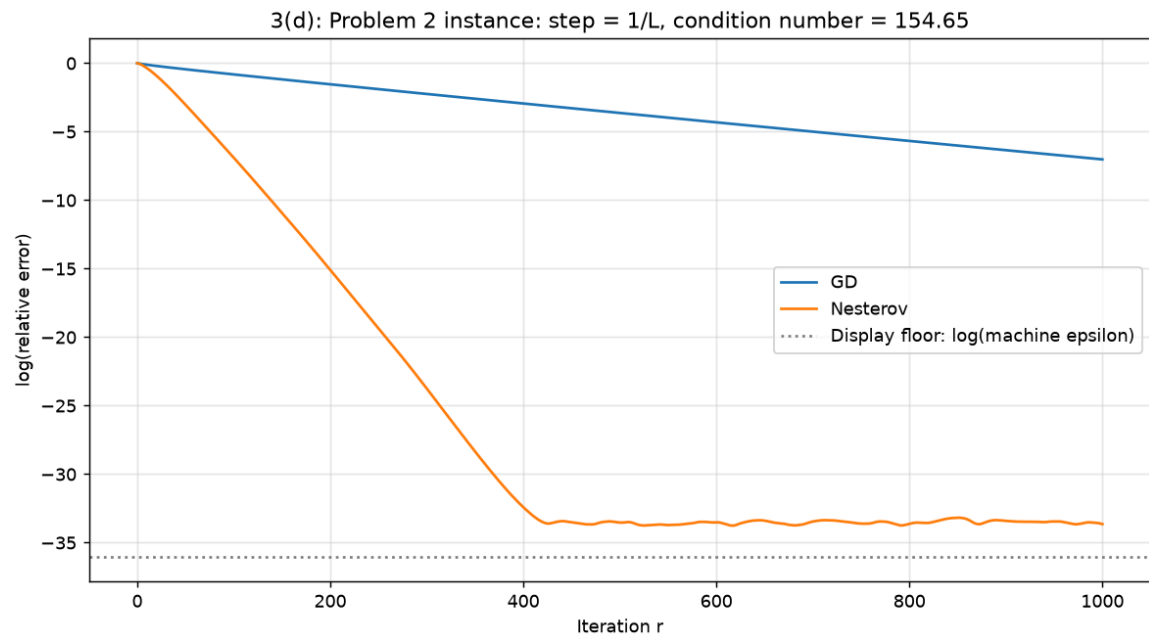
convergence ranges

gradient descent: $0.1 / L \leq \alpha < 1.99 / L$

Nesterov: $0.1 / L \leq \alpha < 1.359 / L$

gradient descent is more robust given its larger stable range.

3(d)



convergence ranges

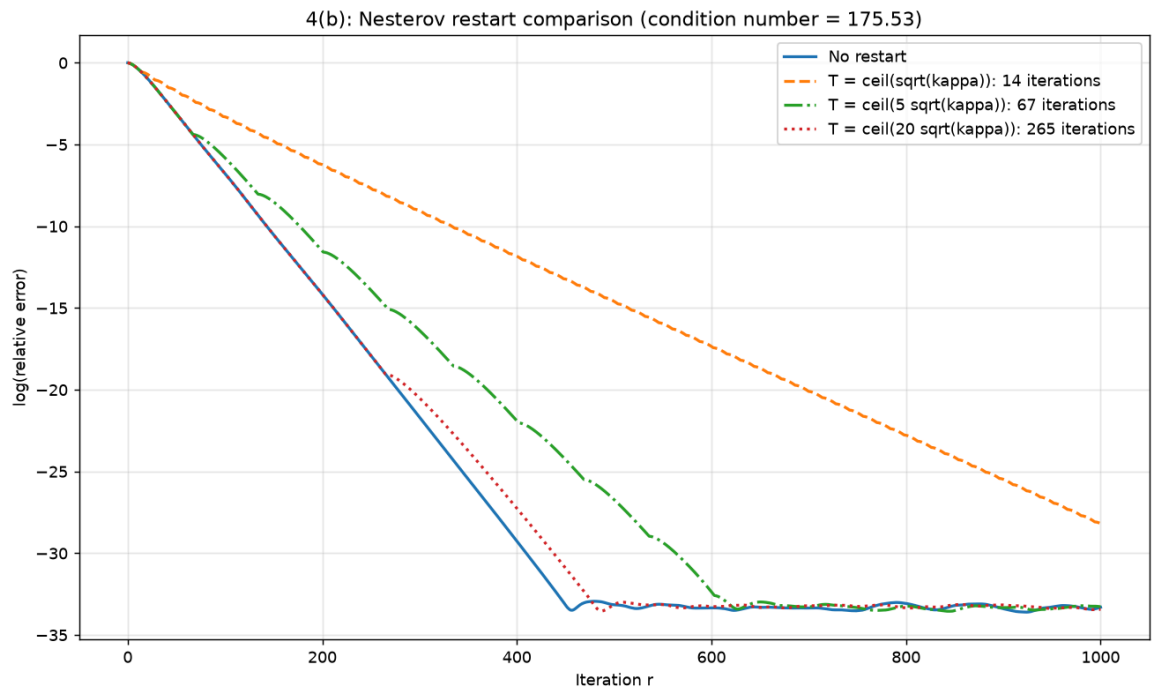
gradient descent: $0.1 / L \leq \alpha < 2 / L$

Nesterov: $0.1 / L \leq \alpha < 1.370 / L$

gradient descent is still more robust given its larger stable range

```
In [14]: # 4(a)
# see problem 1, 2
```

4(b)



4(c)

Since Nesterov is a momentum based algorithm, the more often you 'restart' the more often you reset the 'momentum' of the algorithm, therefore slowing convergence.